UTS
UNIVERSITY OF TECHNOLOGY SYDNEY

Deep Learning

Lecture 3

Plan for Today

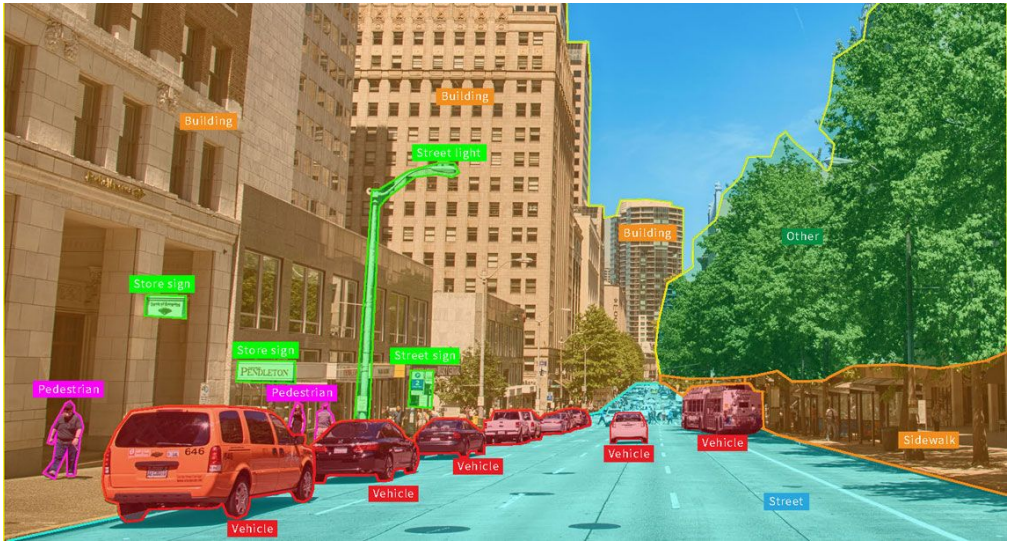
- Computer Vision
- Convolution Neural Networks
- Image Processing Filters
- CNN with PyTorch
- Lab

2

Computer Vision

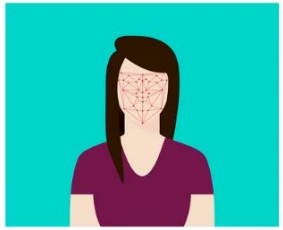
Definition

Computer vision is a field of artificial intelligence (AI) that enables computers and systems to derive meaningful information from digital images, videos and other visual inputs.



Applications

- Optical character recognition (OCR)
- Machine inspection
- Retail (e.g. automated checkouts)
- 3D model building (photogrammetry)
- Medical imaging
- Automotive safety
- Match move (e.g. merging CGI with live actors in movies)
- Motion capture (mocap)
- Surveillance
- Fingerprint recognition and biometrics

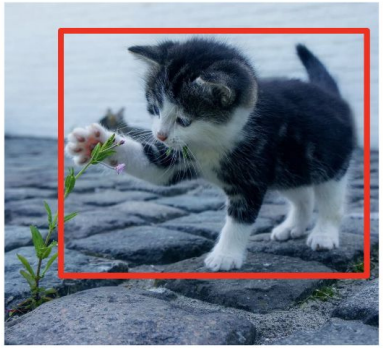


Computer Vision Tasks

Classification

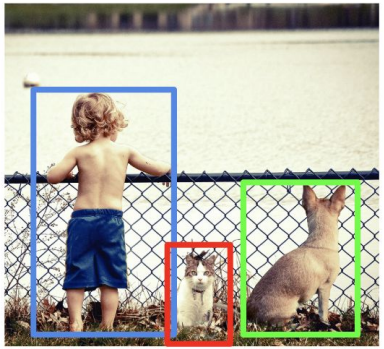
Cat

Classification and Localization



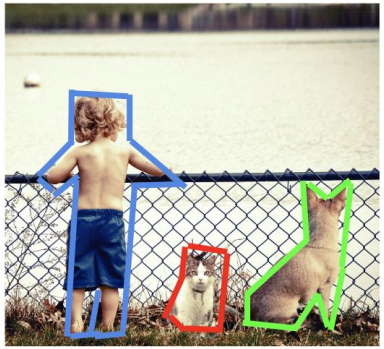
Cat

Object Detection



Kid, cat, dog

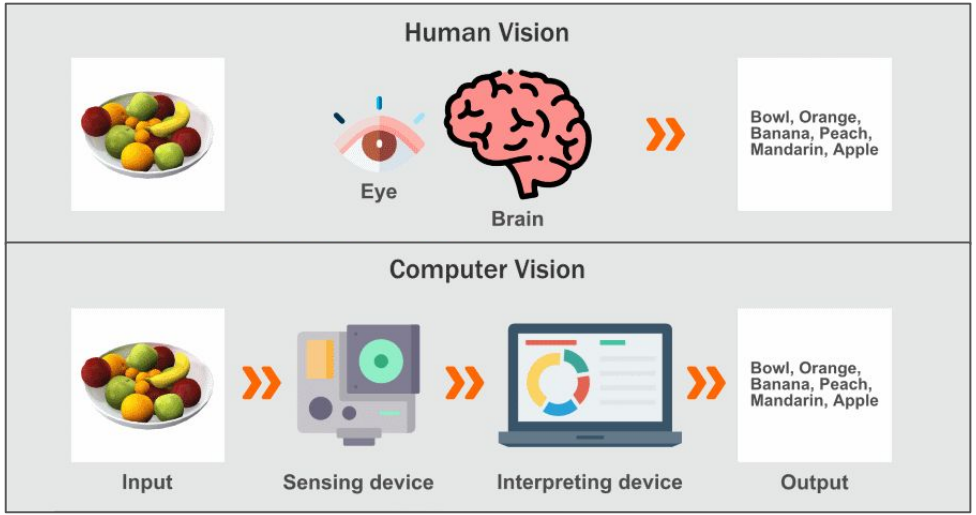
Instance Segmentation

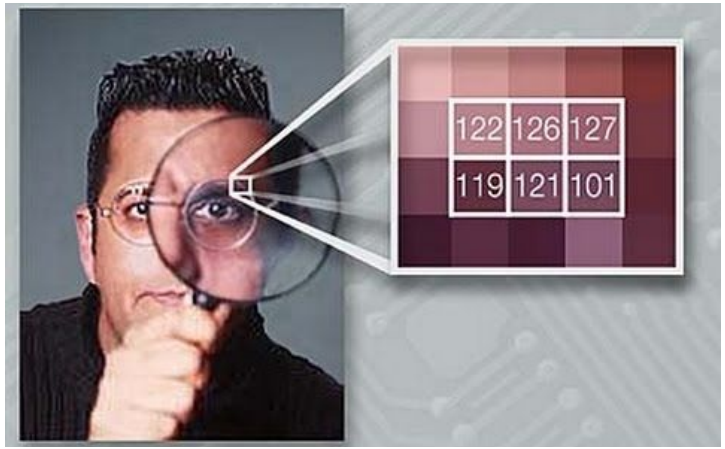


Kid, cat, dog

Digital Images

Computers can’t see like humans. They require digital inputs like matrices. Cameras are used to transform images into a digital format composed of pixels. A pixel is single dot that represents a single point of color. When combined with other pixels, they form the images we see on screens.





Images Dimensions

Digital images are defined by their width and height (like a matrix).

Each of these dimensions are composed of pixels (for example 1024 * 512).

The higher these numbers are the more detailed an image is.

Digital images are unstructured data.

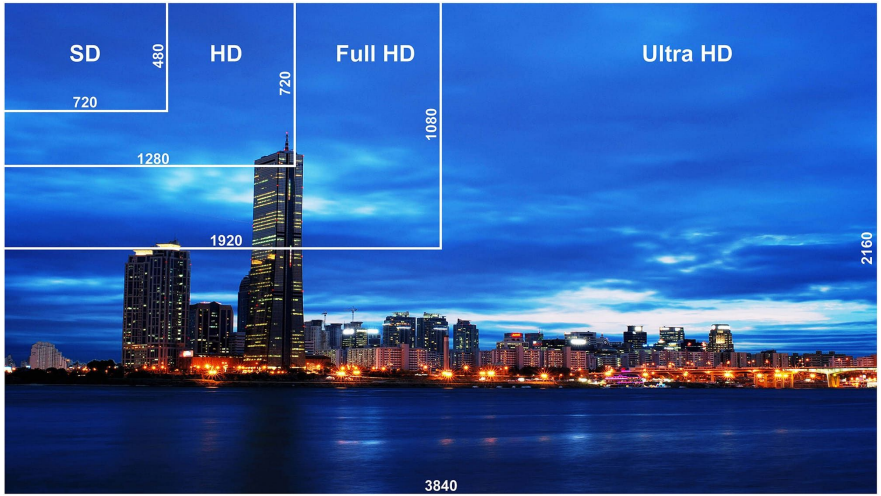
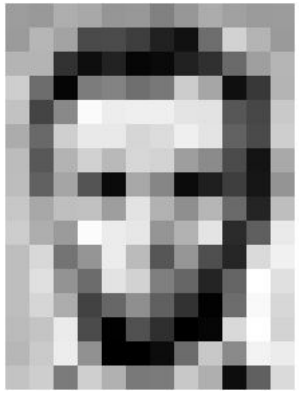


Image Intensity

Each pixel of an image contains a value between 0 and 255.

This value defines the level of intensity of an area of an image (brightness):

- 0 means low intensity (black)
- 255 means high intensity (white)



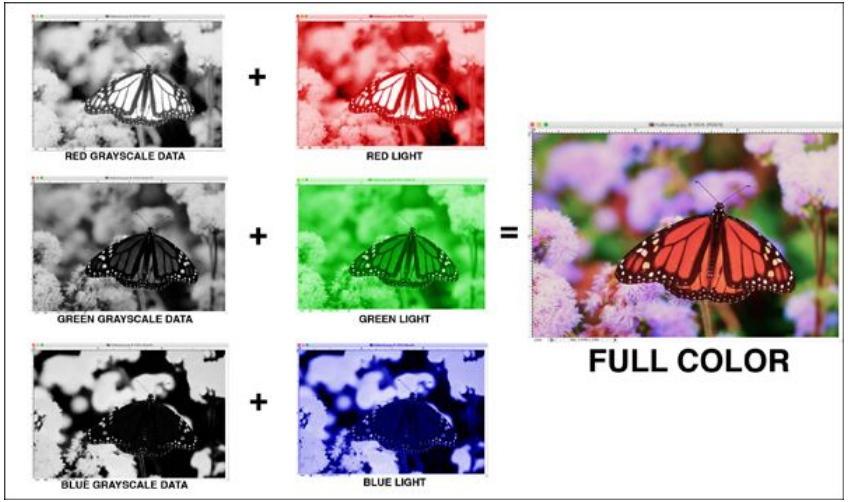
157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

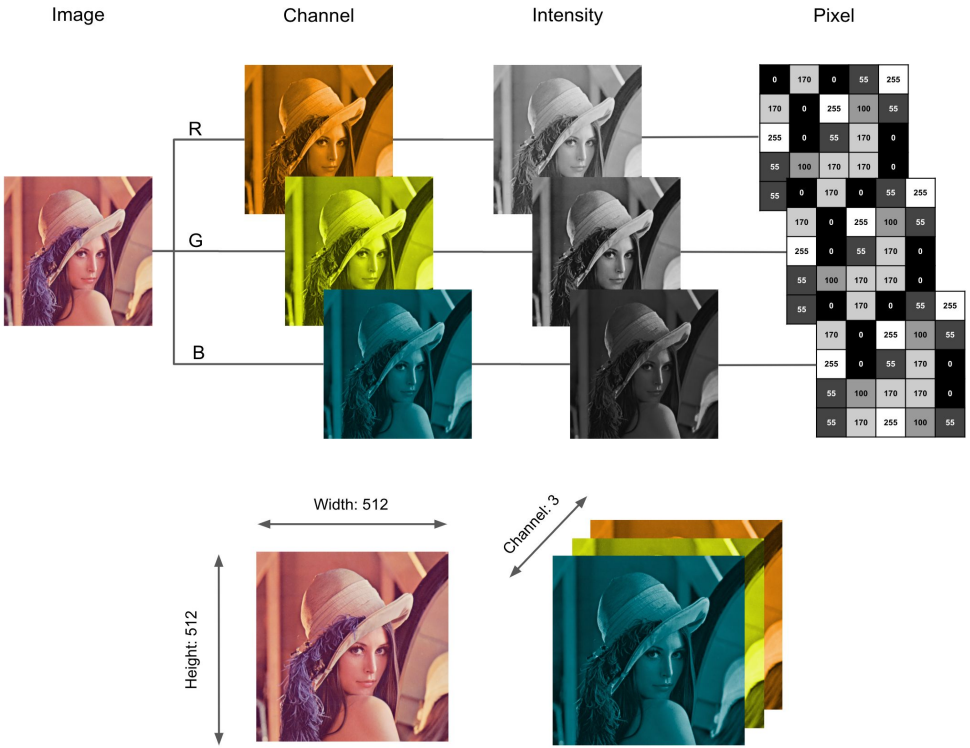
Image Channels

An image can be black and white or have colours. The difference between them are the number of channels used to represent this dimension.

Gray-scale images have only a single channel. While coloured images have 3 different channels (red, green and blue)



Summary



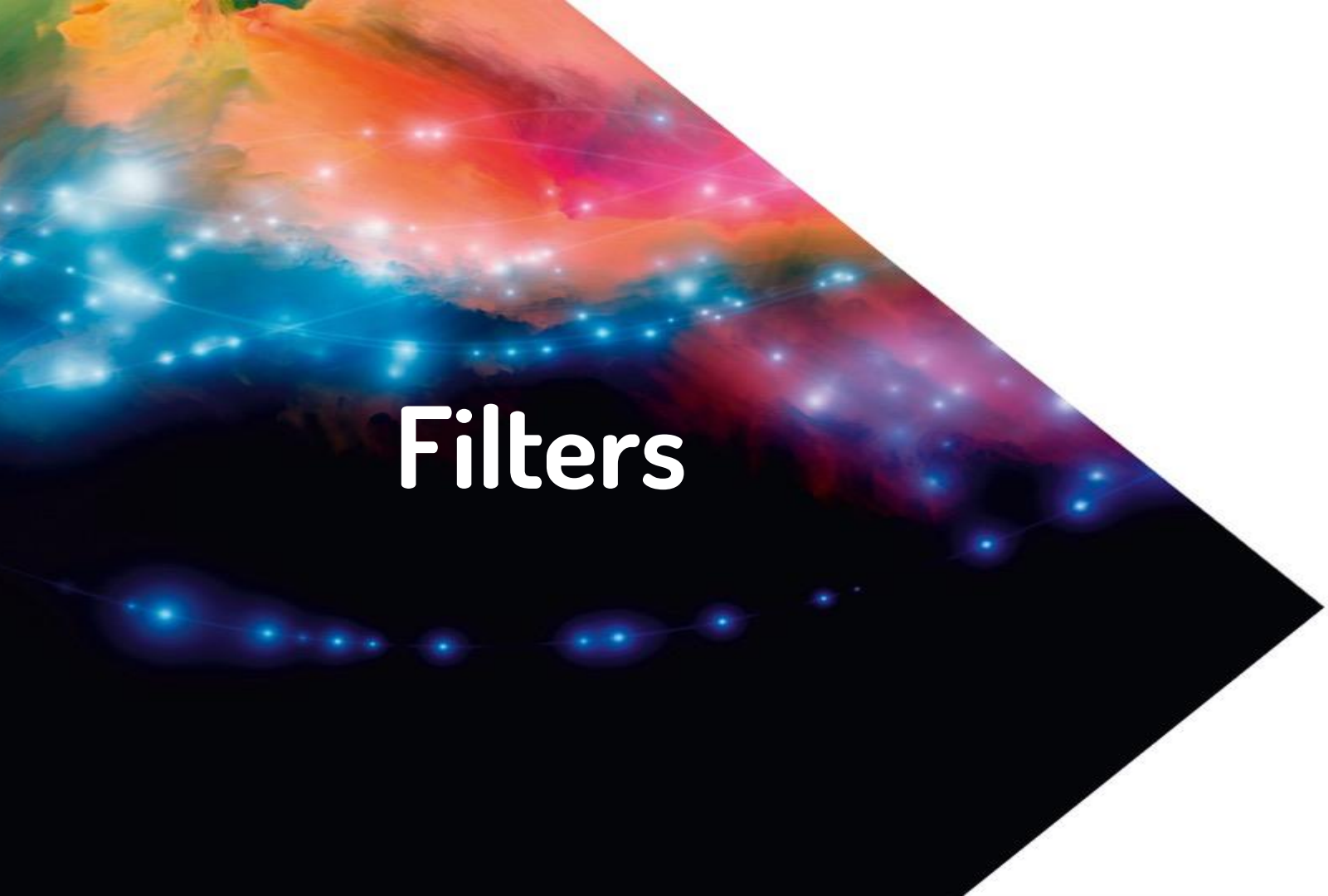
Images are unstructured data

They are defined by their width and height

They are composed of pixels that have value between 0 and 255

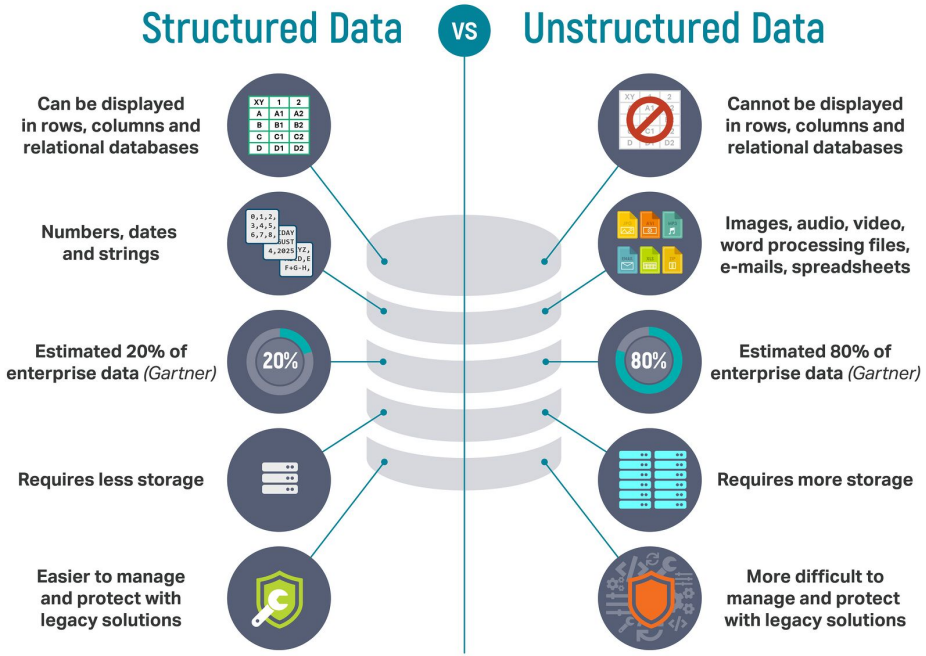
Gray-scale images have only a single channel

Coloured images have 3 different channels (red, green and blue)

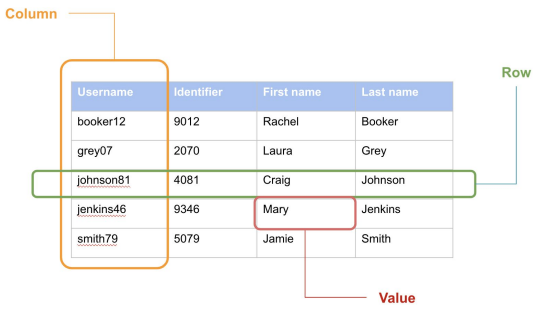


Filters

Structured vs Unstructured Data



Structured



Username	Identifier	First name	Last name
booker12	9012	Rachel	Booker
grey07	2070	Laura	Grey
johnson81	4081	Craig	Johnson
jenkins46	9346	Mary	Jenkins
smith79	5079	Jamie	Smith

Unstructured

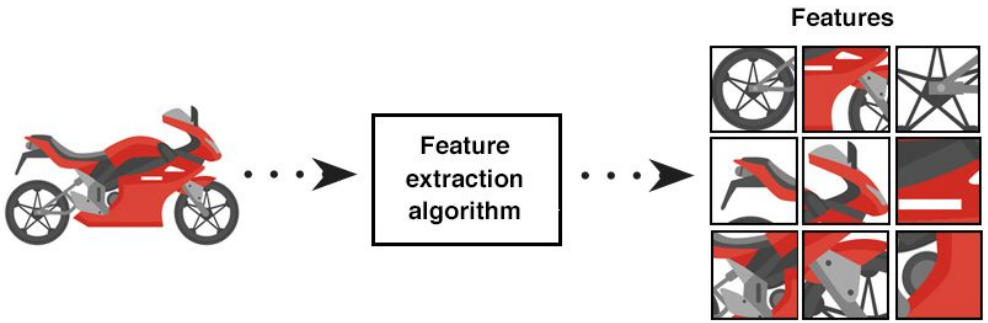
5	10	15	30	0
10	20	30	200	150
100	150	200	15	0
200	150	10	30	0
15	0	0	0	10

Features

Images don’t have features like for structured data with well defined columns and expected values (like age, sex, customer type, average spending, etc) => unstructured data

But images contain a lot of information and patterns.

We need a mechanism that extract features such as colors, textures, shapes, objects, etc.



Filter and Feature Map

In image processing, we used **filters** to extract information from images (feature extraction).

The output of a filter is a feature map which is a derived image with highlighted detected pattern.

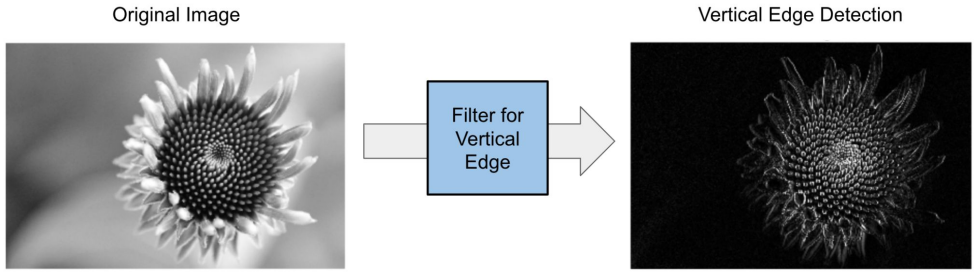
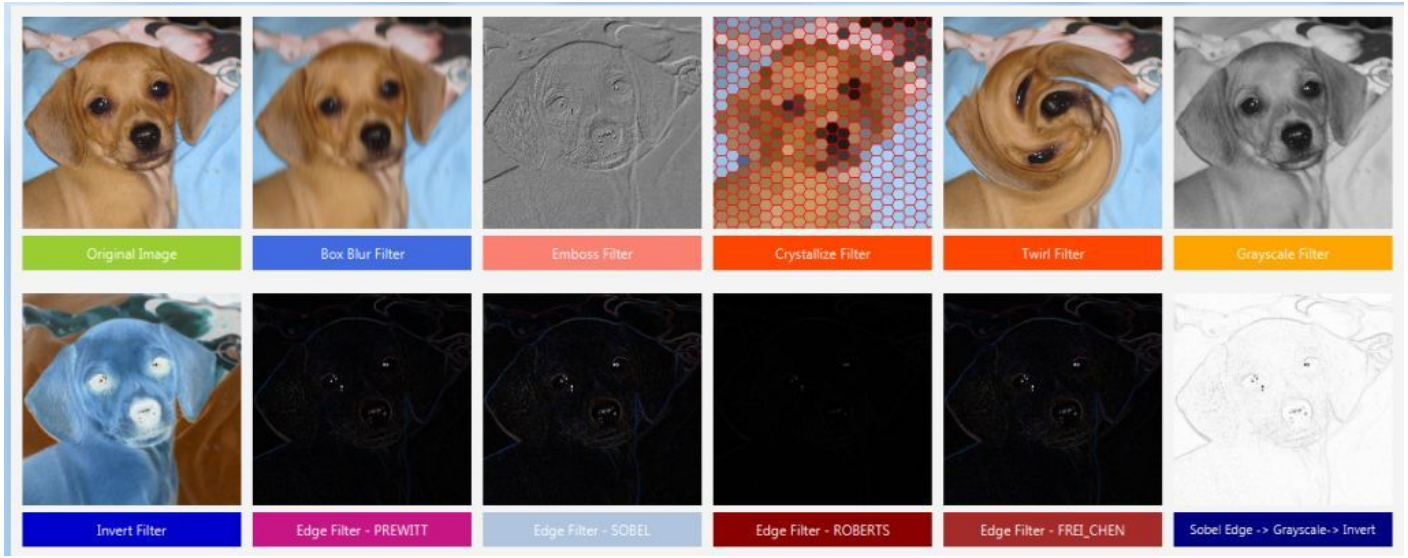
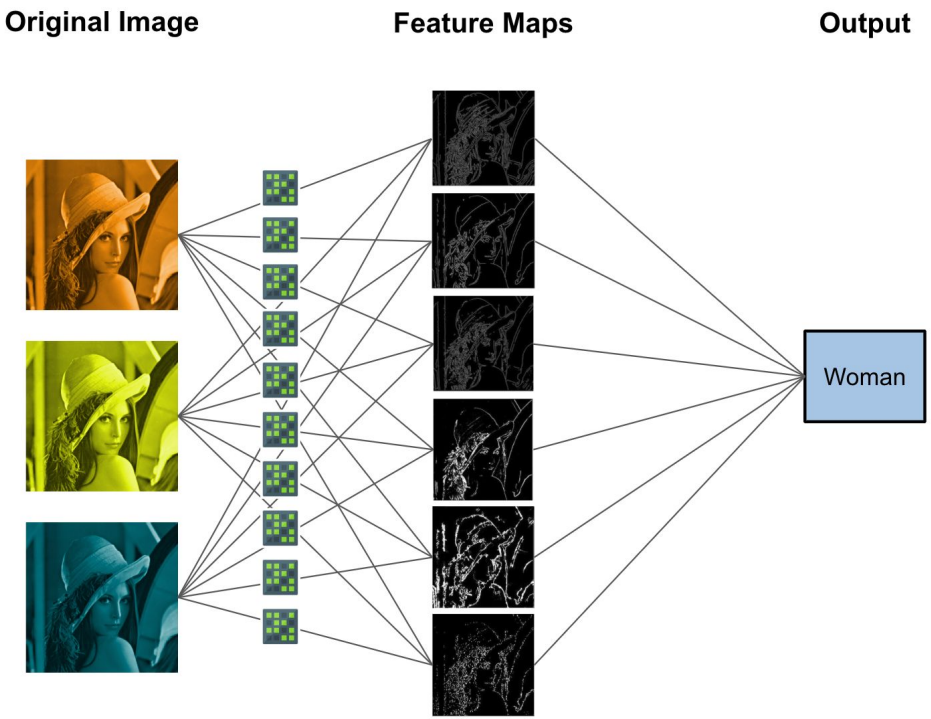


Image Processing Filters



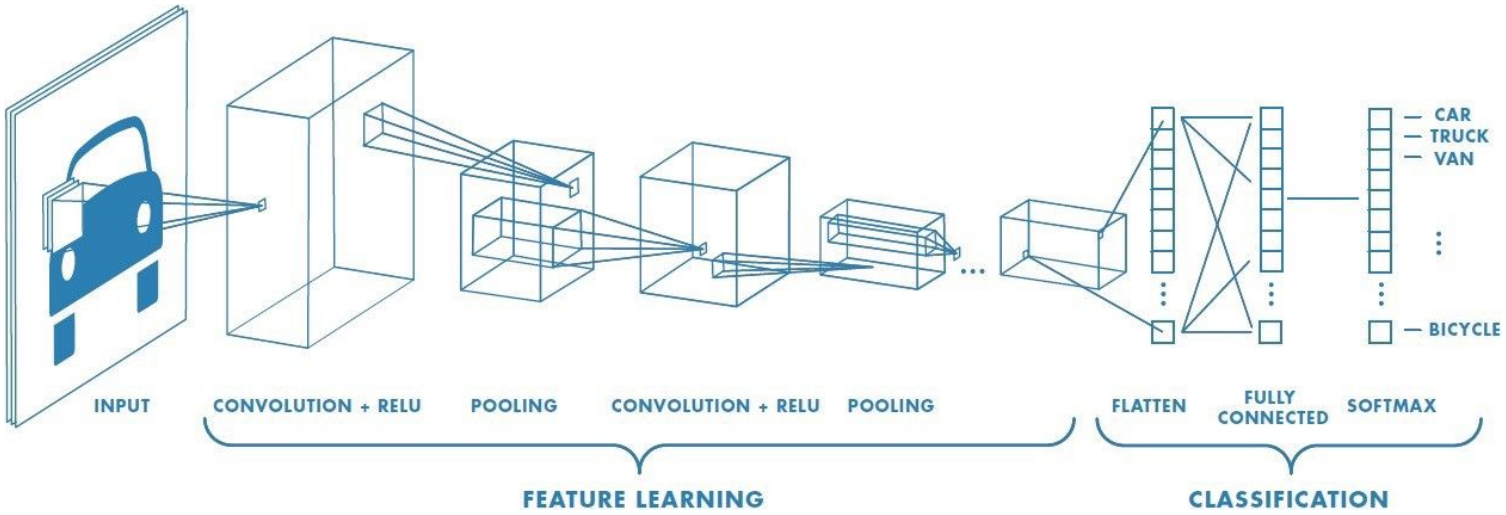
Feature Extraction



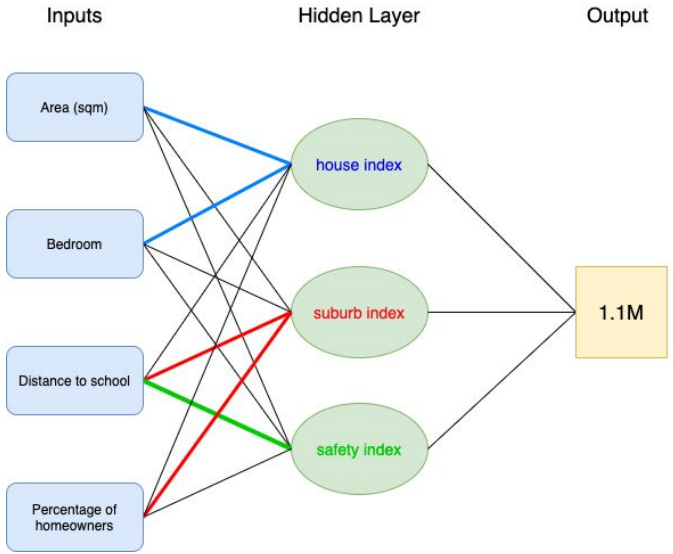
Convolution Neural Networks

Convolution Neural Networks

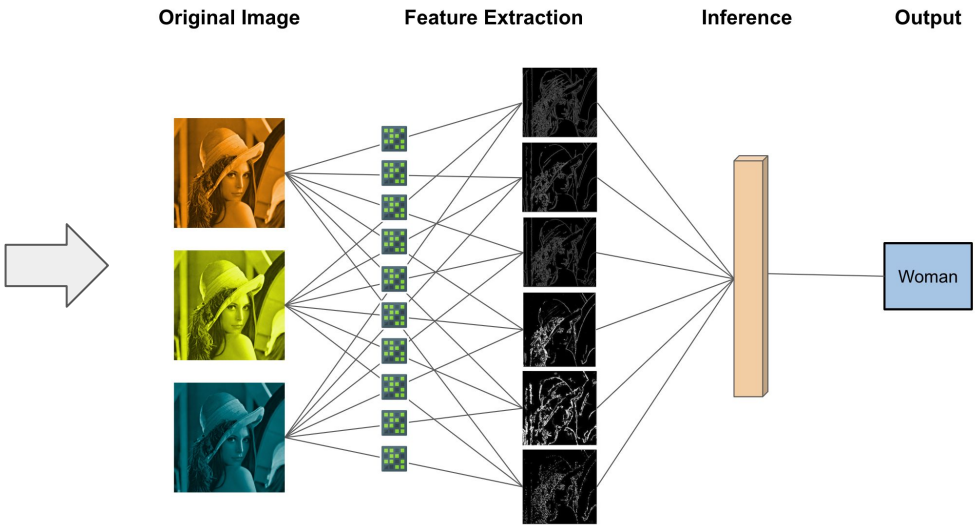
Convolutional Neural Network (CNN) is a class of Artificial Neural Networks that is widely used in various computer vision tasks. CNN is designed to automatically and adaptively learn features from images. It is composed of multiple building blocks, such as convolution layers, pooling layers and fully connected layers.



Extending Neural Networks



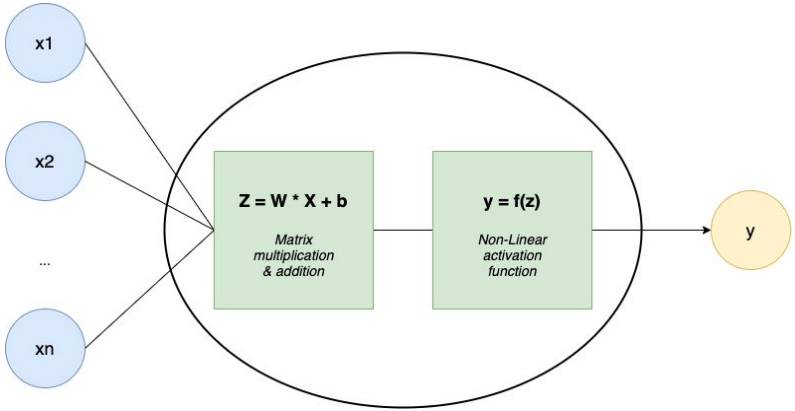
Neural Networks



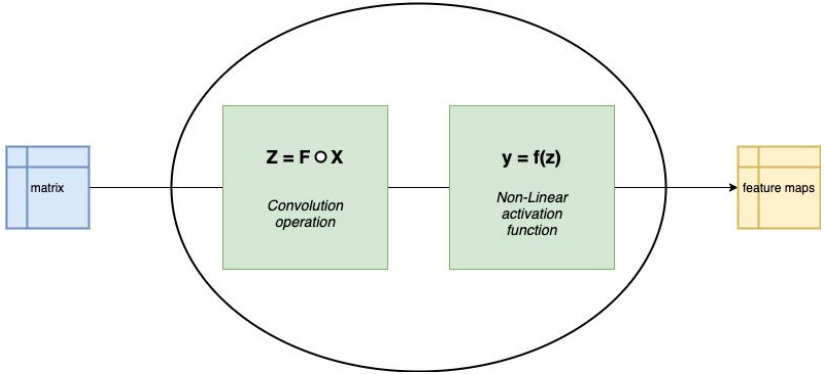
Convolutional Neural Networks

Inside a Neuron

Neural Networks



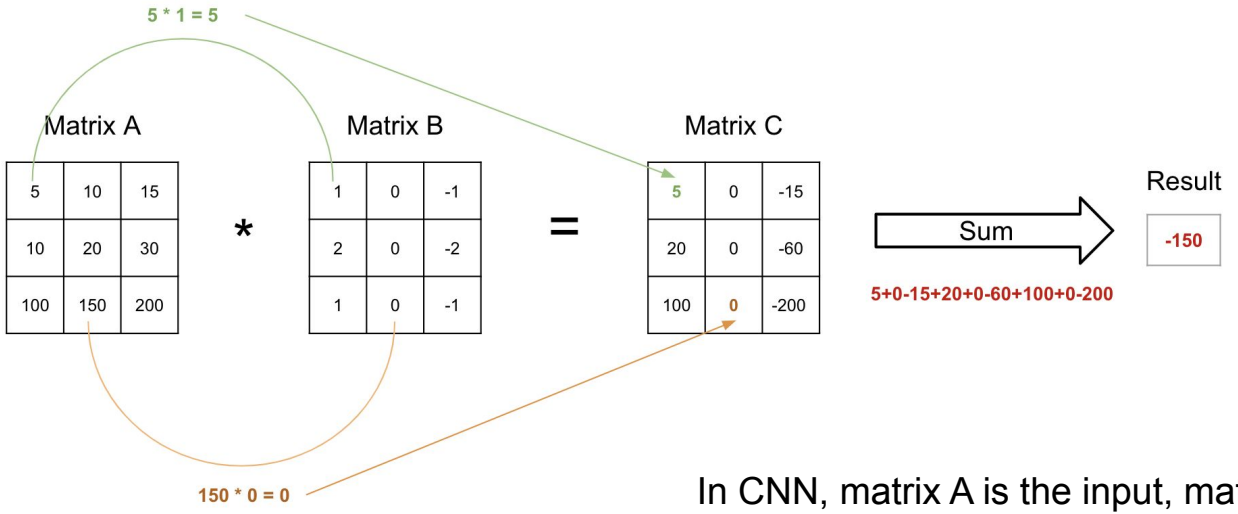
CNN



Convolution Operation

A convolution operation is composed of 3 steps:

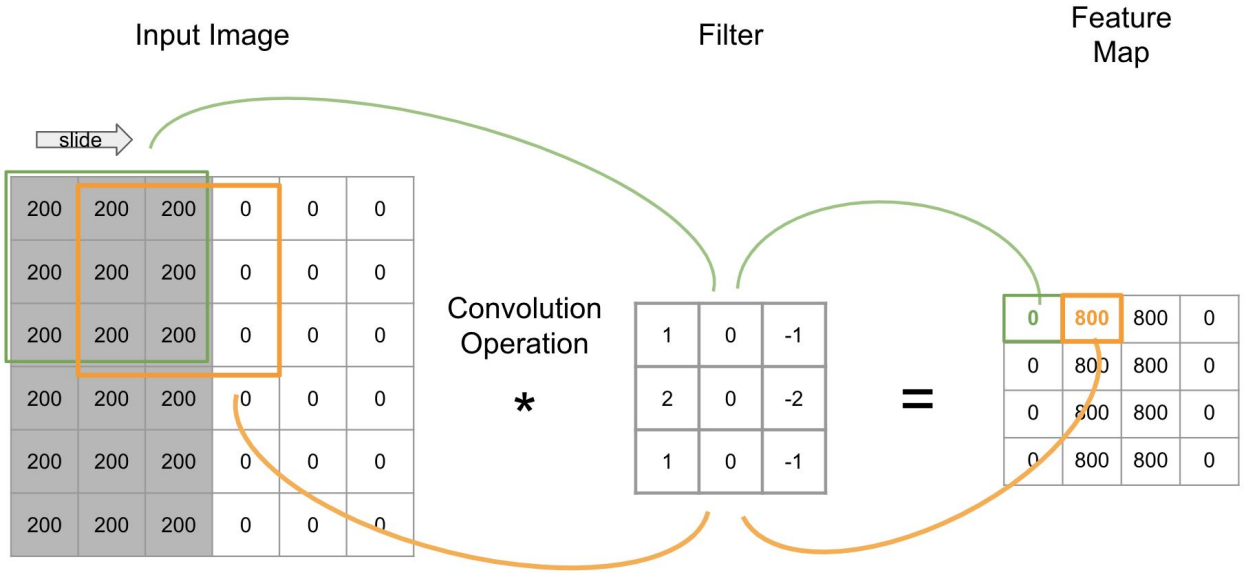
- An element-wise product of 2 matrices (input against a filter)
- A sum of the elements of a matrix
- A stride to process remaining pixels



In CNN, matrix A is the input, matrix B is a filter and matrix C is a feature map

Stride

If a filter is smaller than the input image, the filter will slide in order to process all pixels. The stride will define how many pixels the filter will slide on.



Vertical Filter

Input Image

200	200	200	0	0	0
200	200	200	0	0	0
200	200	200	0	0	0
200	200	200	0	0	0
200	200	200	0	0	0
200	200	200	0	0	0

Convolution Operation

*

Filter

1	0	-1
2	0	-2
1	0	-1

=

Feature Map

0	800	800	0
0	800	800	0
0	800	800	0
0	800	800	0

Vertical Filter

Input Image

200	200	200	200	200	200
200	200	200	200	200	200
200	200	200	200	200	200
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0

Convolution Operation

*

Filter

1	0	-1
2	0	-2
1	0	-1

=

Feature Map

0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0

Vertical Filter

Input Image

Convolution Operation

*

Filter

1	0	-1
2	0	-2
1	0	-1

=

Feature Map

Horizontal Filter

Input Image

200	200	200	0	0	0
200	200	200	0	0	0
200	200	200	0	0	0
200	200	200	0	0	0
200	200	200	0	0	0
200	200	200	0	0	0

Convolution Operation

*

Filter

1	2	1
0	0	0
-1	-2	-1

=

Feature Map

0	0	0	0
0	0	0	0
0	0	0	0
0	0	0	0

Horizontal Filter

Input Image

200	200	200	200	200	200
200	200	200	200	200	200
200	200	200	200	200	200
0	0	0	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0

Convolution Operation

*

Filter

1	2	1
0	0	0
-1	-2	-1

=

Feature Map

0	0	0	0
800	800	800	800
800	800	800	800
0	0	0	0

Horizontal Filter

Input Image

Convolution Operation

*

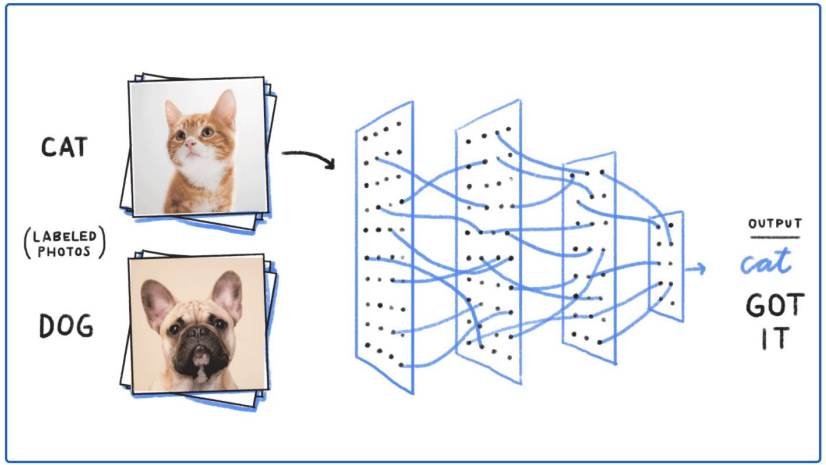
Filter

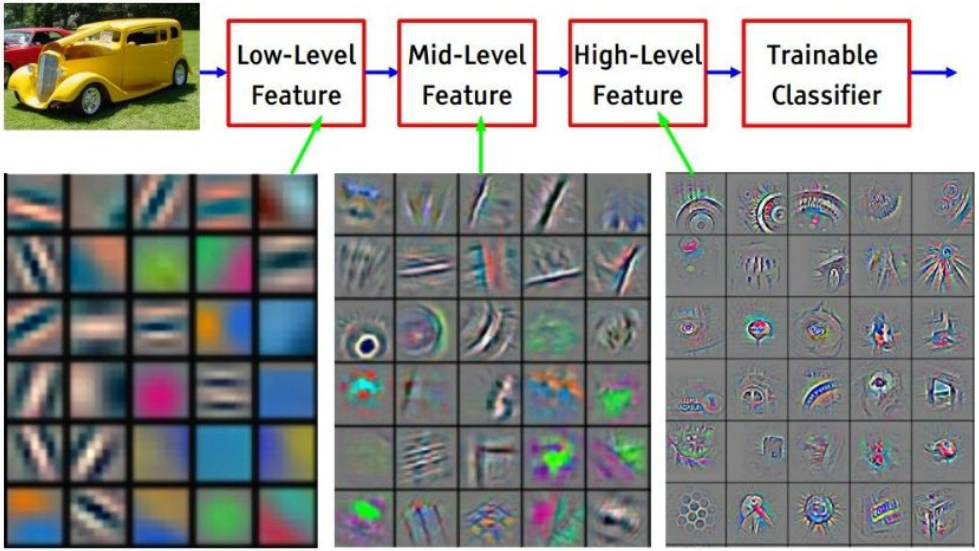
1	2	1
0	0	0
-1	-2	-1

=

Feature Map

Feature Maps Interpretability





Padding

After a convolution the dimensions of the feature map will be smaller than the input image. A way to avoid this is to add padding: add an extra border with value 0 on the image

Input Image

200	200	200	0	0	0
200	200	200	0	0	0
200	200	200	0	0	0
200	200	200	0	0	0
200	200	200	0	0	0
200	200	200	0	0	0

padding

Padded Image

0	0	0	0	0	0	0	0
0	200	200	200	0	0	0	0
0	200	200	200	0	0	0	0
0	200	200	200	0	0	0	0
0	200	200	200	0	0	0	0
0	200	200	200	0	0	0	0
0	200	200	200	0	0	0	0
0	0	0	0	0	0	0	0

Convolution Operation

*

Filter

1	0	-1
2	0	-2
1	0	-1

=

Feature Map

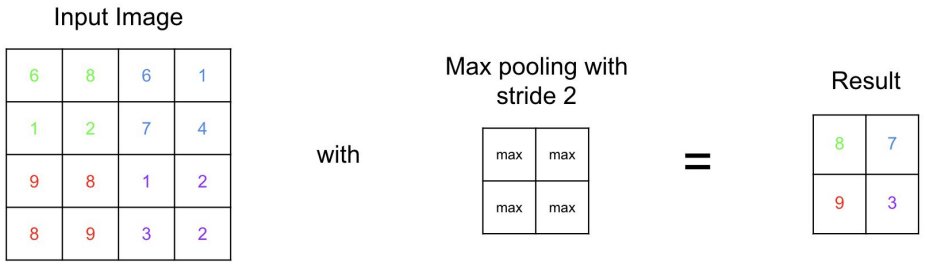
-600	0	600	600	0	0
-800	0	800	800	0	0
-800	0	800	800	0	0
-800	0	800	800	0	0
-800	0	800	800	0	0
-600	0	600	600	0	0

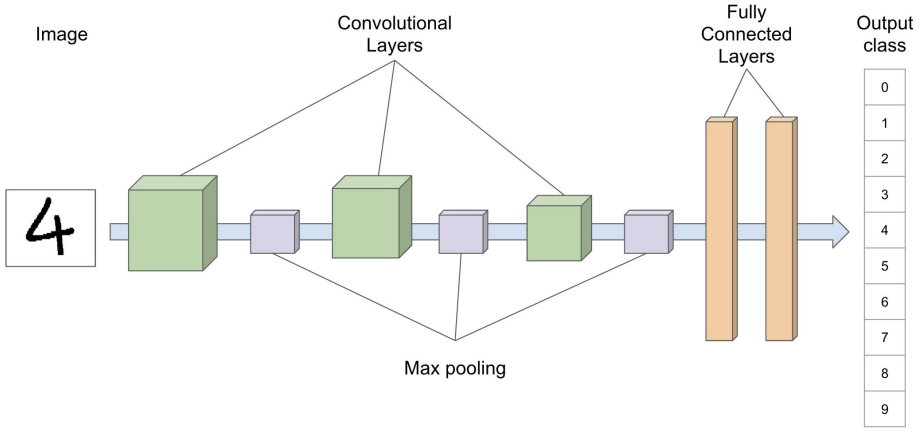
Pooling

Performing convolutions can be quite time-consuming (lot of filters and layers)

We can reduce the feature maps dimensions without losing too much information with pooling

Pooling is like a filter that will extract a measure from the sampled pixels: average, maximum,...





Convolution on an Image

0	0	0	0	0	0	...
0	156	155	156	158	158	...
0	153	154	157	159	159	...
0	149	151	155	158	159	...
0	146	146	149	153	158	...
0	145	143	143	148	158	...
...	...	...	...	...	...	...

Input Channel #1 (Red)

0	0	0	0	0	0	...
0	167	166	167	169	169	...
0	164	165	168	170	170	...
0	160	162	166	169	170	...
0	156	156	159	163	168	...
0	155	153	153	158	168	...
...	...	...	...	...	...	...

Input Channel #2 (Green)

0	0	0	0	0	0	...
0	163	162	163	165	165	...
0	160	161	164	166	166	...
0	156	158	162	165	166	...
0	155	155	158	162	167	...
0	154	152	152	157	167	...
...	...	...	...	...	...	...

Input Channel #3 (Blue)

-1	-1	1
0	1	-1
0	1	1

Kernel Channel #1

1	0	0
1	-1	-1
1	0	-1

Kernel Channel #2

0	1	1
0	1	0
1	-1	1

Kernel Channel #3

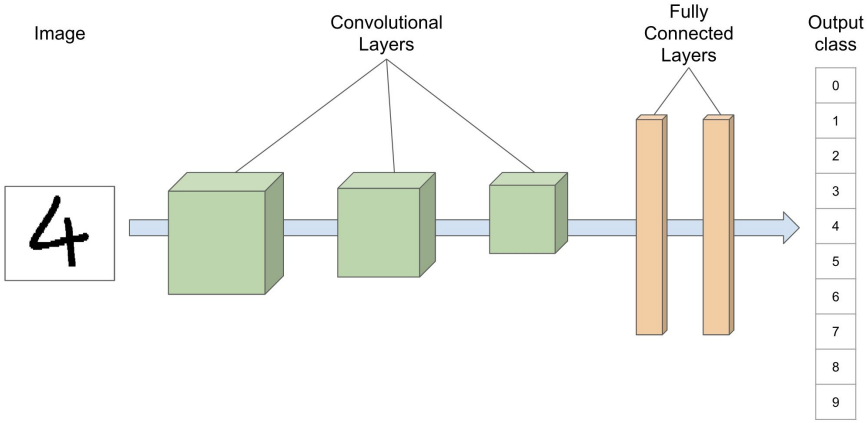
$$308 + -498 + 164 + 1 = -25$$

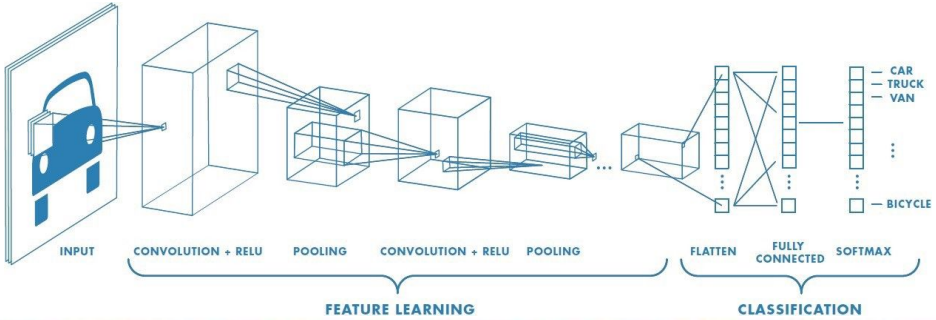
Bias = 1

Output

-25				...
				...
				...
				...
...	...	...	...	...

CNN Architecture





Convolution layers for extracting feature maps

Fully-connected layers (like in NN) for making predictions

Example of a Simple CNN

	Activation Shape	Activation Size	#Parameters
Input	(32,32,3)	3,072	0
CONV1 (f =5, S=1)	(28,28,8)	6,272	608
POOL 1	(14,14,8)	1,568	0
CONV2 (f =5, S=1)	(10,10,16)	1,600	3216
POOL 2	(5,5,16)	400	0
FC1	(120,1)	120	48,120
FC2	(84,1)	84	10,164
Softmax	(10,1)	10	850

CNN with PyTorch

Convolution with PyTorch

nn.Conv2d
nn.functional.conv2d

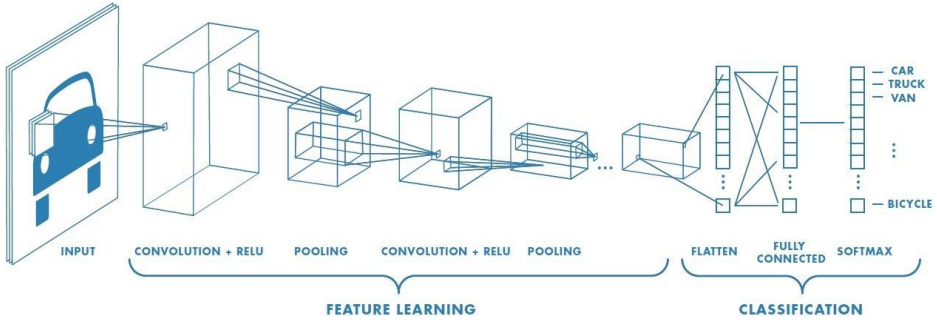
Applies a 2D Convolution operation

- **in_channels** (*int*) - defines the number of channels of the input image.
- **out_channels** (*int*) - defines the number of output channels (ie the number of filters used) of a convolution layer.
- **kernel_size** (*int* or *tuple*) - defines the size of the filters used by a convolution layer.
- **stride** (*int or tuple*, *optional*) - defines the number of pixels used for striding a filter (see explanation in the next section)
- **Padding** (*int or tuple*, *optional*) - defines the number of pixels used for padding an image (see explanation in the next section)

GPU Support: By default, the deep learning framework is installed with GPU support. If your computer has NVIDIA GPUs and has installed CUDA, then you are all set.

nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, stride=1, padding=1),

CNN with PyTorch




```
class CNN(nn.Module):
    def __init__(self):
        super(CNN, self).__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.relu1 = nn.ReLU()
        self.pool1 = nn.MaxPool2d(2, stride=2)

        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.relu2 = nn.ReLU()
        self.pool2 = nn.MaxPool2d(2, stride=2)

       self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(64 * 8 *8, 64)
        self.relu4 = nn.ReLU()
        self.fc2 = nn.Linear(64, number of output class)  # e.g. 100 classes for CIFAR-100
```

```
running_loss = 0.0
    for images, labels in train_dataloader:
        outputs = model(images) # Forward Propagation
        loss = criterion(outputs, labels) # Compute the loss
        losses.append(loss.detach().numpy()) # Keep track of losses
        optimizer.zero_grad()  # clear gradients for training step
        loss.backward() # Back propagation
        optimizer.step() # Update the weights
        running_loss += loss.item()
```

Dataset with PyTorch

PyTorch Documentation: https://pytorch.org/docs/stable/index.html
Torchvision provides many built-in datasets: https://pytorch.org/vision/stable/datasets.html

```
import torchvision
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

train_data = datasets.MNIST(root='./data', train=True, download=True, transform=transform)
test_data = datasets.MNIST(root='./data', train=False, download=True, transform=transform)
```

- root is the path where the train/test data is stored,
- train specifies training or test dataset,
- download=True downloads the data from the internet if it's not available at root.
- transform and target_transform specify the feature and label transformations

Dataset with PyTorch

```
from torch.utils.data import DataLoader
train_dataloader = DataLoader (training_data, batch_size=64, shuffle=True)
test_dataloader = DataLoader (test_data, batch_size=64, shuffle=True)
```

- Dataset retrieves **dataset's features and labels** one sample at a time.
- While training a model, we typically want to pass samples in "minibatches",
- reshuffle the data at every epoch to reduce model overfitting, and use Python's multiprocessing to speed up data retrieval.
- DataLoader is an iterable that abstracts this complexity for us in an easy API.

```
for images, labels in train_dataloader:
    predicted_outputs = model(images) # Forward Propagation to get predicted outcome
    loss = criterion(predicted_outputs, labels) # Compute the loss
    losses.append(loss.detach().numpy()) # Keep track of the losses
    optimizer.zero_grad() # clear gradients for this training step
    loss.backward()   # Back propagation
    optimizer.step()  # Update the weights
    loss_count += loss.item()
```

Train and Test Model

To Train a model general paramaters are

- Number of epochs
- Losses
  - loss = criterion (predicted_outputs, labels)
  - loss.item()

Note: Returns the value of this tensor as a standard Python number

Ref: https://pytorch.org/docs/stable/generated/torch.Tensor.item.html

To Test a model

- Turn on “model.eval()”
- with torch.no_grad()
- Compute corrected values and compare it with total number of samples.

Note: torch.no_grad disables the gradient calculation

Ref: https://pytorch.org/docs/stable/generated/torch.no_grad.html

Note: model.eval() sets the PyTorch model to evaluation mode, disabling dropout, useful for testing and making inference

Time for Lab

42